

UNIDAD 1. SCRIPTS EN SISTEMAS OPERATIVOS

Es un **fichero de texto normal que consta de bloques de código** formados por líneas de comandos que se ejecutan secuencialmente para automatizar determinadas tareas.

El usuario debe tener permisos de ejecución.

1. CREACIÓN

- **#!** → Sha Bang → **#!/bin/bash** → Nos asegura que siempre será lanzada por bash
- Aquí continuaría las líneas de comandos que contenga el script
- Guardamos el script con el nombre_script.sh

2. EJECUCIÓN

- **sh nombre_script.sh** → Si no le damos permisos de ejecución
- **./nombre_script.sh** → Si le damos permisos de ejecución con *chmod a+x nombre_script.sh*

Si queremos lanzar el script desde cualquier lugar o usuario, debemos darle permisos *chmod 777 nombre_script.sh* y moverlo al directorio **/usr/bin**.

3. BUENA PRAXIS

Una buena praxis en scripts es tener las siguientes características:

- Encabezado → Autor, fecha y descripción
- Debe ejecutarse sin errores y realizar la tarea asignada
- La lógica interna debe estar bien definida
- No realizará acciones innecesarias
- Debe ser reutilizable
- Contener comentarios

Principio KISS → Keep it short and simple → Breve y simple.

4. DEPURACIÓN

OPCIÓN SET	OPCIÓN BASH	DESCRIPCIÓN
noexec	-n	Comprueba sintaxis pero no ejecuta los comandos
verbose	-v	Imprime comandos antes de usar
xtrace	-x	Imprime los comandos y las distintas expansiones

Lo utilizaremos **después del sh** -(depuración) nombre_script.sh

- sh -n nombre_script.sh
- sh -v nombre_script.sh
- sh -x nombre_script.sh

5. ENTRECOMILLADOS

- ‘ ’ → Comillas simples → Interpretado literalmente
- “ ” → Comillas dobles → Interpretado literalmente y caracteres especiales como \$
- ` ` → Tildes invertidas → Ejecuta lo que tenga en su interior

6. METACARACTERES SINTÁCTICOS

METACARÁCTER	DESCRIPCIÓN
;	Separador entre órdenes que se ejecutan secuencialmente
	Separación entre órdenes que forman parte de un cauce. Empieza a ejecutarse lo que haya más a la izquierda
\	Posibilita escribir líneas de órdenes en varias líneas. Se utiliza para comandos largos
()	Se utilizan para aislar órdenes separadas por ; o . Lo que esté dentro se considera una única orden.
{ }	Crea un bloque de código ejecutado en el propio intérprete
&	Indicador de trabajo en segundo plano
	Equivalente a OR. Ejecuta si lo precedente falla
& &	Equivalente a AND. Ejecuta si lo precedente no falla

7. METACARACTERES DE ARCHIVO

METACARÁCTER	DESCRIPCIÓN
?	Iguala cualquier carácter simple
*	Iguala cualquier secuencia de cero o más caracteres
[]	Designa un carácter o rango que son igualados por un simple carácter. Para determinar un rango se separa con un guion (-)
{ }	Abrevia un conjunto de palabras
~	Abrevia el camino absoluta o ruta del directorio /home

8. VARIABLES

Es un **espacio en memoria** donde puede guardar un valor para usarlo más adelante en el script. Existen 3 tipos de variables en bash:

- **Usuario** → Son las definidas por el usuario → fecha=\$(date)
- **Entorno** → Son las definidas por el sistema → echo \$HOME
- **Especiales** → Son las propias del script → echo \$#

Hay 2 áreas de memoria en el Shell para almacenar variables:

- **Área local de datos** → Se asigna por defecto, son privadas y se puede acceder a ellas
- **De entorno** → Almacenadas y disponibles para los procesos hijos

8.1 Variables de Entorno

VARIABLE	DESCRIPCIÓN	VALOR POR OMISIÓN
DISPLAY	Aparecen las salidas X-Windows	
EDITOR	Editor usado por defecto	
FUNCNAME	Nombre de la función en ejecución	La modifica el Shell
HOME	Directorio personal de la cuenta	Lo define root
HOSTNAME	Nombre de la máquina	
IFS	Separador de campos de entrada	ESP, TAB, NL
LANG	Idioma de los mensajes	
LINENO	Número de línea del script	La modifica el Shell
LOGNAME	Nombre del usuario que ejecuta el Shell	Activado por login
OLDPWD	Directorio de trabajo anterior	Lo modifica el Shell
PATH	Camino de búsqueda de órdenes	Según el sistema
PPID	Identificador del proceso padres	Lo modifica el Shell
PS1...PS4	Puntos indicativos primario, secundario, selectivo y de errores	
PWD	Directorio de trabajo actual	Lo modifica el Shell
SHELL	Path del programa intérprete	El propio Shell
TERM	Tipo de terminal	
USER	Nombre del usuario	

- **export** → Cuando le pasamos el nombre de una variable, ésta es trasladada desde el área local de datos al entorno
- **set** → Informa de los nombre y valores de todas las variables del Shell en el área local de datos y el entorno
- **env** → Informa de los valores y nombre de todas las variables de entorno
- **unset** → Borra el valor de las variables del área local de datos (sin argumentos)

8.2 Variables Especiales

VARIABLE	DESCRIPCIÓN
\$\$	Identificador del proceso (PID)
\$*	Cadena con el contenido completo de los parámetros recibidos
\$@	Como '\$*' pero trata cada parámetro como una palabra diferente
\$#	Número de parámetros
\$?	Código de retorno del último comando
#!	Último identificador del proceso ejecutado en segundo plano
\$_	Valor del último argumento del comando ejecutado previamente

8.3 Expresiones

Bash **permite utilizar una gran variedad de expresiones en el desarrollo de scripts.**

- **let** → Se utiliza para las expresiones aritméticas → Representan operaciones con números enteros o binarios
- **bc** → Lenguaje de cálculo de precisión arbitraria → Se utiliza para realizar operaciones con números reales

8.4 Utilidad *basename*

Muestra por la salida estándar (**stdout**) el nombre del fichero que se le pase como argumento, eliminando el prefijo del directorio que pueda contener.

9. CONDICIONALES → IF

Permite **ejecutar un bloque de código solo si se cumple una condición.**

ALTERNATIVA	SINTAXIS	DESCRIPCIÓN
Simple	if [condición] then bloque fi	Si la condición se cumple, ejecuta lo que se encuentre en bloque dentro de then
Doble	if [condición] then bloque 1 else bloque 2 fi	Si la condición se cumple, ejecuta lo que se encuentre de bloque 1 dentro de then, si no, ejecuta lo que esté en el bloque 2 dentro de else
Múltiple	if [condición 1] then bloque 1 elif [condición 2] then bloque 2 else bloque 3 fi	Si se cumple la condición 1, ejecuta lo que se encuentre en bloque 1 dentro de then, si no, pasa a la condición 2, si se cumple, ejecuta el bloque 2. En caso contrario ejecuta lo que esté en bloque 3

9.1 Sintaxis

1. **if** → Nombre de la estructura condicional seguido de [condición]
2. **[condición]** → Debe tener espacios después y antes de los corchetes
3. **then** → Indica que hacer si se cumple la condición
4. **else** → Indica qué hacer si no se cumple la condición
5. **fi** → Es el cierre de la estructura condicional

9.2 Operadores Alfanuméricos

CONDICIÓN	DESCRIPCIÓN
variable1 = variable2	Verdadero si variable1 es igual a variable2
variable1 != variable2	Verdadero si variable1 no es igual a variable2
variable1 < variable2	Verdadero si variable1 es menor a variable2
variable1 > variable2	Verdadero si variable1 es mayor a variable2
-n variable1	Verdadero si variable1 no es nulo (tiene un valor)
-z variable1	Verdadero si variable1 es nulo (no tiene un valor)

9.3 Operadores Numéricos

CONDICIÓN	DESCRIPCIÓN
número1 -eq número2	Verdadero si número1 es igual a número2
número1 -ne número2	Verdadero si número1 no es igual a número2
número1 -lt número2	Verdadero si número1 es menor a número2
número1 -gt número2	Verdadero si número1 es mayor a número2
número1 -le número2	Verdadero si número1 es menor o igual a número2
número1 -ge número2	Verdadero si número1 es mayor o igual a número2

9.4 Operadores de Ficheros

CONDICIÓN	DESCRIPCIÓN
-a fichero	Verdadero si fichero existe
-b fichero	Verdadero si fichero existe y es un dispositivo de bloques
-c fichero	Verdadero si fichero existe y es un fichero de caracteres
-d fichero	Verdadero si fichero existe y es un directorio
-e fichero	Verdadero si fichero existe y es de cualquier tipo
-f fichero	Verdadero si fichero existe y es un fichero regular
-h fichero -L fichero	Verdadero si fichero existe y es un enlace simbólico
-k fichero	Verdadero si fichero existe y tiene activo el sticky bit
-p fichero	Verdadero si fichero existe y es un cauce (pipe)
-s fichero	Verdadero si fichero existe y su tamaño es mayor a cero
-S fichero	Verdadero si fichero existe y es un socket de comunicaciones
-r fichero	Verdadero si fichero existe y se puede leer
-t descriptor	Verdadero si el descriptor de fichero está abierto y asociado a una terminal
-w fichero	Verdadero si fichero existe y se puede escribir

-x fichero	Verdadero si fichero existe y se puede ejecutar
fichero1 -nt fichero2	Verdadero si fichero1 es más nuevo que fichero2
fichero1 -ot fichero2	Verdadero si fichero1 es más viejo que fichero2

9.5 Operadores para Concatenar Expresiones

SINTAXIS	DESCRIPCIÓN
if [expresión1] && [expresión2]	Se ejecuta si expresión1 y expresión2 son verdaderas
if [expresión1] [expresión2]	Se ejecuta si expresión1 o expresión2 son verdaderas
If ! [expresión1]	Se ejecuta si expresión1 no es verdadera

10. CONDICIONALES → CASE

Permite **ejecutar un bloque de código si el valor de la expresión es uno u otro**. Suele utilizarse para crear “menús”

10.1 Sintaxis

```

case $variable in
valor1)                → Comienzo de la estructura
    bloque1            → Valor que se introduce
    ;;                → Código a ejecutar
valor2)                → Cierre del primer valor
    bloque2
    ;;
*)                    → Valor por defecto que no está determinado
    bloque por defecto
    ;;
esac                  → Cierre de la estructura

```

VALOR	DESCRIPCIÓN
*	Equivale a cualquier grupo. Se utiliza para el caso por defecto
?	Equivale a cualquier carácter simple
[...]	Define una clase de caracteres. Si se separa con un guion (-)m se refiere a un rango de caracteres
 	Separación entre caracteres alternativos

11. OPERACIONES ALGEBRAICAS

Permiten hacer cálculos numéricos con números enteros y expresiones aritméticas

11.1 Sintaxis

expr arg1 operación arg2 → Ejemplo: resultado=\$(expr \$a + \$b)

Si queremos indicar que el resultado tenga decimales, usaremos **scale= n° decimales** y utilizaremos la calculadora de linux (bc -l)

resultado=\$(echo "scale= n° decimales; \$a + \$b | bc -l)

11.2 Operaciones Numéricas

OPERACIÓN	DESCRIPCIÓN
expr num1 + num2	Devuelve la suma de num1 + num2
expr num1 - num2	Devuelve la resta de num1 - num2
expr num1 * num2	Devuelve la multiplicación de num1 * num2
expr num1 / num2	Devuelve la división de num1 / num2
expr num1 >= num2	Devuelve 0(true) si num1 >= num2
expr num1 > num2	Devuelve 0(true) si num1 > num2
expr num1 <= num2	Devuelve 0(true) si num1 <= num2
expr num1 < num2	Devuelve 0(true) si num1 < num2
expr num1 != num2	Devuelve 0(true) si num1 es distinto de num2
%	Módulo (Resto de una división)

Si queremos poner entre paréntesis una operación deberemos indicarlo con \

Ejemplo → 4* (4+3)/2 → **expr 4 * \ (4+3 \) / 2**

12. BUCLES → FOR

Se utiliza para **repetir un conjunto de comando varias veces**, recorriendo una lista de valores, archivos o incluso rango numéricos.

12.1 Sintaxis

```
for variable in conjunto
do
    bloque
    [break | continue]
done
```

- **for** → Comienzo del bucle
- **variable in** → Toma cada valor de un conjunto, uno por uno
- **conjunto** → Todos los valores de una lista o conjunto
- **do** → Inicio del bloque de comandos
- **break | continue** → Rompe el bucle | salta al siguiente valor de la lista
- **done** → Final del bloque y/o bucle

13. BUCLES → WHILE

Se utiliza para **repetir un bloque de código mientras la condición sea verdadera**.

13.1 Sintaxis

```
while [ condición ]  
do  
    bloque  
done
```

- **while** → Inicio del bucle
- **[condición]** → Condición que tiene que cumplirse
- **do** → Inicio del bloque de comandos
- **done** → Fin del bloque y/o bucle

También puede optarse por leer las líneas de un fichero, en ese caso utilizaremos la siguiente sintaxis:

```
while read linea          * En este caso, leería línea a línea el archivo.txt  
do  
    bloque  
done <archivo.txt
```

14. BUCLES → UNTIL

Se utiliza para **repetir un bloque de código siempre que la condición sea falsa**.

```
until [ condición ]  
do  
    bloque  
done
```

- **until** → Inicio del bucle
- **[condición]** → Condición que tiene que cumplirse
- **do** → Inicio del bloque de comandos
- **done** → Fin del bloque y/o bucle

15. PASO DE PARÁMETROS

PARÁMETROS	DESCRIPCIÓN
\$1	Devuelve el 1er parámetro pasado al script o función
\$2	Devuelve el 2do parámetro
\$3	Devuelve el 3er parámetro
\$*	Devuelve todos los parámetros separados por espacio
\$#	Devuelve el número de parámetros que se han pasado
\$0	Devuelve el parámetro 0, es decir, el nombre del script o de la función

16. SELECT

Para su funcionamiento, **select hace uso de PS3** (prompt selectivo) donde su valor por defecto es #?.

Se suele utilizar como “**menú**”. Hay que darle permisos *chmod a+x* al script y ejecutarlo como *./script.sh*.

17. FUNCIONES

Una función es un **conjunto de instrucciones** que se agrupan para realizar una tarea concreta y que es fácilmente reutilizable. Se recomienda que su nombre sea lo más descriptivo posible y que no contengan la letra ‘ñ’.

17.1 Sintaxis

```
nombre_funcion () {  
bloque  
[return [valor]]  
}
```

- **nombre_funcion () {** → Comienzo de la función
- **return [valor]** → Termina la ejecución e indica un código de salida
- **}** → Final de la función

18. COMANDO → DIALOG

Crea **interfaces o ventanas** tipo “**popup**” interactivas en modo texto dentro de la terminal.

COMANDO	DESCRIPCIÓN
--msgbox	Muestra un mensaje informativo
--yesno	Pregunta Sí/No
--inputbox	Permite escribir texto
--passwordbox	Entrada oculta para contraseñas
--menu	Muestra un menú de opciones. Se utiliza \ al final de cada opción.
--checklist	Lista con múltiples opciones seleccionables
--radiolist	Lista con solo una opción seleccionable
--textbox	Muestra el contenido de un archivo
--infobox	Muestra información temporal
--calendar	Selector de fecha
--gauge	Barra de progreso
--form	Múltiples campos de entrada
--timebox	Selector de hora
--tailbox	Muestra un archivo en tiempo real

18.1 Sintaxis

En casi todos los casos la sintaxis es parecida:

dialog --comando “Texto de cabecera” X Y → Donde ‘X’ es el alto e ‘Y’ es el ancho